

Tasca S4.01

BigQuery Avançat & Analytics Engineering

Com a Analytics Engineer, la teva responsabilitat no és només lliurar dades, sinó assegurar l'escalabilitat i l'eficiència de la plataforma.

En aquest sprint final, redissenaràs l'arquitectura de les dades per optimitzar costos (Partitioning), introduiràs lògica analítica avançada (Window Functions) i modelaràs estructures complexes (Arrays) per tancar el cicle amb l'automatització

Nivell 1: Entorn i Ingesta Híbrida (Code-First)

Escenari:

El quadre de comandament financer tarda massa a carregar i el cost és inacceptablement alt. S'ha identificat que el problema crític són els informes que creuen vendes amb regions geogràfiques.

Exercici 1: Consulta sobre Taula no Optimitzada (Diagnòstic)

El Country Manager d'**Alemanya** necessita revisar urgentment les transaccions del dia **12 de març de 2022**.



Tasca:

- » 1. Escriu la consulta que uneix (JOIN) transaccions i companyies.
- » 2. Filtra els resultats per la data indicada i el país "Germany".
- » 3. Sense executar la consulta, realitza un "Dry Run" (auditoria de costos).

👁️ *Observació: Fixa't que BigQuery llegeix gairebé tota la taula tot i demanar només un dia (Full Table Scan).*

» Consultes

```
1
2 SELECT t.*
3 FROM `sprint3_silver.transactions_clean` AS t
4 INNER JOIN `sprint3-analytics-rcarrera1.sprint3_silver.companies_clean` AS c
5 ON t.business_id = c.company_id
6 WHERE t.declined = 0 AND c.country = 'Germany' AND EXTRACT(DATE FROM t.timestamp)= '2022-03-12';
7
8
```

✓ Esta consulta procesará 11.85 MB cuando se ejecute.

```
1
2 SELECT t.*
3 FROM `sprint3_silver.transactions_clean` AS t
4 INNER JOIN `sprint3-analytics-rcarrera1.sprint3_silver.companies_clean` AS c
5 ON t.business_id = c.company_id;
6
7
8
```

✓ Esta consulta procesará 11.85 MB cuando se ejecute.

Observacions: hem fet dues consultes, una filtrant per país i data i l'altra sense filtres. Observem que processen el **mateix número de bytes**. Això és degut al fet que a la primera consulta BigQuery també ha d'escanejar tots els registres de la taula per veure si compleixen la condició.

Exercici 2: Re-arquitectura i Optimització de l'Emmagatzematge (Partition & Cluster)

Escenari:

Després del diagnòstic de l'exercici anterior, hem confirmat que filtrar la taula de transaccions per data o per empresa provoca un **Full Table Scan** (lectura completa de la taula), cosa que és ineficient. Per solucionar-ho a la capa de consum (*Gold*), materialitzarem una versió optimitzada de la taula de fets.



Tasca:

Pas 1: Generació de Dades Recents (**Mocking Data**)

Pas 2: Creació de la Taula Optimitzada (**Partitioning & Clustering**)



Entrega:

- » El codi SQL utilitzat per al Pas 1.
- » El codi DDL (CREATE OR REPLACE TABLE...) utilitzat al Pas 2.
- » Una captura de pantalla dels "Detalls" de la nova taula **fact_transactions_optimized** on es pugui comprovar que està particionada i agrupada en clústers correctament.

» Pas 1: Generació de Dades Recents (Mocking Data)

Crea una taula intermèdia anomenada `sprint3_silver.transactions_recent` a partir de la taula `sprint3_silver.transactions_clean`. El teu objectiu és mantenir totes les columnes, però substituir el timestamp original per un de nou, generat aleatòriament perquè caigui dins dels últims 50 dies.

```
1  
2  
3 CREATE OR REPLACE TABLE `sprint3_silver.transactions_recent` AS  
4 SELECT * EXCEPT (timestamp),  
5 TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL CAST(RAND()*50 AS INT64) DAY) AS timestamp  
6 FROM `sprint3_silver.transactions_clean`;  
7
```

✓ Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Chat

Información del trabajo

Resultados

Detalles de la ejecución

Gráfico de ejecución



Esta declaración reemplazó la tabla denominada `transactions_recent`.

Observacions: Utilitzem **SELECT * EXCEPT** per seleccionar tots els camps menys el camp **timestamp**. Generem el camp **timestamp** restant de la data actual un número aleatori entre 0 i 50.

» Pas 2: Creació de la Taula Optimitzada (Partitioning & Clustering)

Ara, crea la teva taula definitiva **sprint3_gold.fact_transactions_optimized** a partir de les dades recents que acabes de generar a **transactions_recent**. Has de construir la sentència DDL per configurar la taula amb les següents estratègies físiques d'emmagatzematge: Partitioning i Clustering.

```
1
2 CREATE OR REPLACE TABLE `sprint3_gold.fact_transactions_optimized`
3 PARTITION BY DATE(timestamp)
4 CLUSTER BY business_id
5 OPTIONS (
6   description = "Taula particionada per data i dins de cada partició ordenada per codi empresa"
7 ) AS
8 SELECT *
9 FROM `sprint3_silver.transactions_recent`;
10
11
```

✓ Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Chat

Información del trabajo

Resultados

Detalles de la ejecución

Gráfico de ejecución



Esta declaración creó una nueva tabla con el nombre fact_transactions_optimized .

Observacions:

- **PARTITION BY DATE (timestamp):** : divideix la taula per la data. Això permetrà que les consultes que filtrin per dia només llegeixin la partició necessària i no hagin d'escanejar tota la taula per fer el filtre. Això reduirà el cost de la consulta.→ **Partitioning**
- **CLUSTER BY business_id:** ordena les dades dins de cada partició pel camp business_id. Això farà que els filtres per companyia siguin molt més ràpids i menys costosos.→ **Clustering**

» COMPROVACIÓ DE LA DEFINICIÓ DE LA TAULA

sprint3-analytics-rcarrera1 / Datasets / sprint3_gold / Tables / **fact_transactions_optimized**

☆ fact_transacti... Consulta Chat Abrir en ▾ + Compartir ▾ Copiar

i Esta es una tabla particionada. [Learn more](#)

Esquema	Detalles	Vista previa	Explorador de tablas	Vista previa	Estadísticas	Linaje
Tipo de tabla	Particionada					
Particionada por	DAY					
Particionada en el campo	timestamp					
Vencimiento de la partición	60 días					
Filtro de partición	No es obligatorio					
Agrupada por	business_id					

Observacions: la taula de fets **fact_transactions_optimized** està particionada per dia i dins de cada partició està ordenada pel camp business_id.

Exercici 3: La Prova del Cotó (Benchmark)

Escenari:

Ja has creat la taula **sprint3_gold.fact_transactions_optimized** amb les estratègies de **Particionament** i **Clustering**. Ara toca demostrar a l'equip la millora de rendiment i l'estalvi de costos que has aconseguit. Simularem una petició típica de negoci: "Volem veure totes les transaccions dels últims 30 dies".

🧩 Tasca:

L'objectiu és clar i directe: **aplicar exactament la mateixa consulta a dues taules diferents i comparar-ne el cost computacional**. Construeix una consulta SQL que seleccioni totes les columnes (SELECT *) i filtri les dades dels últims 30 dies.

⚠️ **NO executis les consultes encara**. En aquest exercici només utilitzarem el validador de BigQuery ("Dry Run", a dalt a la dreta de l'editor) per visualitzar els **Bytes processats** de manera gratuïta.

👉 Entrega:

- » El codi SQL base que has dissenyat.
- » Les dues captures de pantalla on es vegi la diferència de "Bytes processats" al validador.
- » Un breu text calculant quin és el % **d'estalvi** que has aconseguit .

» Pas 1 (Taula no optimitzada)

Apunta la teva consulta a la taula `sprint3_silver.transactions_recent`. Observa el validador i fes una captura de pantalla dels bytes que processaria.

```
2 SELECT *
3 FROM `sprint3_silver.transactions_recent`
4 WHERE DATE(timestamp) >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY);
5
6
7
```

✓ Esta consulta procesará 11.85 MB cuando se ejecute.

Observacions: la consulta no optimitzada processa **11.85MB** ja que BigQuery ha d'escanejar el 100% de la taula per fer el filtre.

» Pas 2 (Taula optimitzada)

Sense canviar res de la lògica de la teva consulta, modifica només el FROM perquè ara apunti a la taula particionada `sprint3_gold.fact_transactions_optimized`. Observa com canvia el validador i fes la segona captura.

```
1
2 SELECT *
3 FROM `sprint3_gold.fact_transactions_optimized`
4 WHERE DATE(timestamp) >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY);
5
6
7
```

✅ Esta consulta procesará 7.21 MB cuando se ejecute.

Observacions: la consulta optimitzada processa **7,21MB** ja que BigQuery només ha de llegir les particions dels dies demanats.

Fem càlculs: $(11,85 - 7,21) * 100 / 11,85 = 39,15$

Això ens indica que hem aconseguit aproximadament un **40% d'estalvi** en el MB processats.

Exercici 4: Smart Caching (Vistes Materialitzades)

Escenari:

El Director General té un quadre de comandament que mostra les "**Vendes Totals per Dia**". Ell (i 50 managers més) refresquen aquest gràfic constantment.

Problema:

Si utilitzes una consulta normal o una Vista estàndard, cada vegada que algú refresca la pàgina, BigQuery recalcula la suma de milions de files. És lent i estàs pagant pel mateix càlcul una vegada i una altra.

Solució:

Utilitzar una **Vista Materialitzada**. Aquesta tecnologia guarda el *resultat* del càlcul a la memòria cau i només l'actualitza si entren dades noves (incremental).



Tasca:

- » Crea una **Vista Materialitzada** anomenada **sprint3_gold.mv_daily_sales** que mostri les **Vendes Totals per Dia**.



Entrega:

- » Codi per Crear la **Vista Materialitzada**.
- » Fes una consulta que mostri la vista creada i fes una captura de pantalla que mostri en el seu conjunt la vista materialitzada creada, amb la consulta executada i els bits processats.

» Vendes Totals per Dia

Creació de la Vista Materialitzada

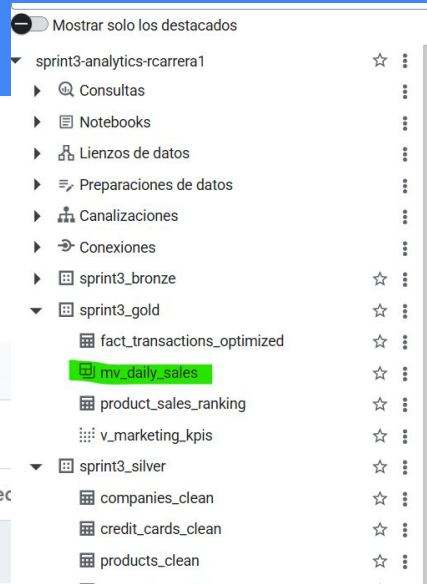
```
1 CREATE MATERIALIZED VIEW `sprint3_gold.mv_daily_sales` AS (  
2 SELECT  
3     DATE(t.timestamp) AS sale_date,  
4     SUM(t.amount) AS total_amount,  
5     COUNT(t.transaction_id) AS total_transactions  
6 FROM  
7     `sprint3_gold.fact_transactions_optimized` AS t  
8 WHERE t.declined = 0  
9 GROUP BY sale_date);  
11
```

Se completó la consulta

Resultados de la consulta

Información del trabajo Resultados Detalles de la ejecución Gráfico de ejecución

Esta declaración creó una nueva vista materializada denominada mv_daily_sales.



Consulta sobre la Vista

2 SELECT sale_date, ROUND(total_amount,2), total_transactions
3 FROM `sprint3\_gold.mv\_daily\_sales`
4 ORDER BY sale_date;
5
6

Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo	Resultados	Visualización	JSON	Detalles
ID de trabajo	sprint3-analytics-rcarrera1:EU.bquxjob_2a05f3ea_19eb6e54f0			
Usuario	rosercarrera13@gmail.com			
Ubicación	EU			
Hora de creación	11 jun 2026, 3:35:43 p.m. UTC+2			
Hora de inicio	11 jun 2026, 3:35:43 p.m. UTC+2			
Hora de finalización	11 jun 2026, 3:35:43 p.m. UTC+2			
Duración	0 s			
Bytes procesados	1.2 KB			
Bytes facturados	1.0 MB			
Milisegundos de ranura	32			
Prioridad del trabajo	INTERACTIVE			

Observacions: Utilitzem **Vistes Materialitzades** per estalviar costos.

Característiques:

- guarden els resultats de la consulta a la memòria cau de manera quan fem consultes sobre les vistes processem pocs bytes, en el nostre cas 1,2MB.
- s'actualitzen de forma automàtica quan hi ha algun canvi a la taula original.

Molt útils per dades que cal consultar molt sovint.

Resultat de la consulta

Fila	sale_date	f0_	total_transactions
1	2026-04-22	255210.76	980
2	2026-04-23	532051.78	2017
3	2026-04-24	532944.0	2054
4	2026-04-25	511811.3	1937
5	2026-04-26	528428.1	2012
6	2026-04-27	517111.09	1986
7	2026-04-28	502699.11	1977

Nivell 2: SQL Analític Avançat

Escenari:

La direcció comença a fer preguntes que no es resolen amb un simple GROUP BY. Necessitem anàlisi de tendències i comportament d'usuari.

Exercici 1: Perfilat de Clients VIP

(Mètriques Agregades amb CTEs)

Escenari

L'equip de **Màrqueting** vol analitzar el comportament dels nostres millors clients per dissenyar l'estratègia del pròxim any. Definim "VIP" com aquells amb una despesa acumulada superior a 500€. Necessiten un informe que, per a cada VIP, mostri el seu nom, contacte i el seu patró de compra: quantes vegades ha comprat, quant gasta de mitjana i quina va ser la seva compra rècord.

✚ Tasca:

» **1. Crea una CTE anomenada VIP_Stats que agrupi per usuari i calculi:**

- La Despesa Total (SUM).
- La Quantitat de Transaccions (COUNT).
- El Tiquet Mitjà (AVG), arrodonit a 2 decimals.
- La Compra Màxima (MAX).
- Filtre: Mantén només aquells la Despesa Total dels quals sigui > 500.

» **Creua la CTE amb users_combined per obtenir les dades personals.**

Requisits de Sortida:

- Columnes: user_id, nom_complet, email, num_compres, tiquet_mig, max_compra, total_gastat.
- Ordenat per total_gastat descendent.

» Consulta clients VIP

```

1
2 WITH VIP_Stats AS(
3 SELECT
4     t.user_id,
5     ROUND(SUM(t.amount),2) AS total_amount,
6     COUNT(t.transaction_id) AS total_transactions,
7     ROUND(AVG(t.amount),2) AS average_amount,
8     MAX(t.amount) AS max_amount
9 FROM `sprint3_gold.fact_transactions_optimized` AS t
10 WHERE t.declined = 0
11 GROUP BY t.user_id
12 HAVING total_amount > 500
13 )
14 SELECT u.user_id, CONCAT(u.name, ' ', u.surname) AS name, u.email, vs.total_transactions, vs.average_amount, vs.max_amount, vs.total_amount
15 FROM VIP_Stats AS vs
16 INNER JOIN `sprint3_silver.users_combined` AS u
17 ON vs.user_id = u.user_id
18 ORDER BY vs.total_amount DESC;
19

```

✓ Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

[Chat](#) [Guardar los resultados](#) [Abrir](#)

Información del trabajo		Resultados	Visualización	JSON	Detalles de la ejecución		Gráfico de ejecución	
Fila	user_id	name	email	total_transactions	average_amount	max_amount	total_amount	
1	289	Dxwgi Hwcru	dxwgi.hwcru@example.com	91	446.91	554.71	40669.16	
2	318	Bnyr Astuw	bnyr.astuw@example.com	86	449.0	615.51	38614.39	
3	455	Fwfl Phylgnmhnd	fwfl.phylgnmhnd@example.com	66	522.6	724.44	34491.27	
4	417	Fqatil Rdwcjvoa	fqatil.rdwcjvoa@example.com	73	421.87	675.78	30796.3	
5	185	Molly Gilliam	donec@outlook.couk	107	274.72	659.31	29394.79	
6	285	Brcxprnj Lnpna	brcxprnj.lnpna@example.com	70	417.85	630.52	29249.51	
7	193	Minerva Wilkins	aliquam.adipiscing@yahoo.edu	67	380.37	646.0	25485.05	

Observacions:

Utilitzem una **CTE (VIP_Stats)** per calcular les diferents **mètriques** per cada usuari.
Després enllacem amb la taula d'usuaris per capturar els **atributs** (nom complet, email...)-

Exercici 2: Anàlisi de Tendències

(Window Functions sobre Vistes)

Escenari:

La **Direcció Financera** vol monitoritzar la "velocitat de vendes" diària. Necessiten un informe que compari el rendiment de cada dia contra el dia anterior per detectar caigudes brusques o pics de creixement (**Day-over-Day Growth**).



Estratègia d'Optimització:

Per a aquest càlcul **NO** has d'anar a la taula de fets original ni recalcul ar sumes. Com a bon **Data Engineer**, has d'utilitzar la Vista Materialitzada **sprint3_gold.mv_daily_sales** que vas crear a l'exercici anterior. Això garanteix que l'anàlisi de tendències sigui instantània i no consumeixi recursos de còmput innecessaris.



Tasca:

Crea una consulta sobre la vista materialitzada que generi un informe amb les següents 4 columnes:

- » 1. **Data**: La data de venda.
- » 2. **Vendes_Avui**: El total venut aquell dia.
- » 3. **Vendes_Ahir**: El total venut el dia anterior (usant funcions de finestra).
- » 4. **Diff_Percentual**: El percentatge de creixement o decreixement respecte al dia anterior, arrodonit a 2 decimals.



Pista Tècnica:

Utilitza la funció **LAG(camp) OVER (ORDER BY data)** per llegir el valor de la fila anterior sense necessitat de fer JOINS complexos.

» Informe Day-over-day Growth

```

1
2 WITH amount_previous_day AS (
3   SELECT sale_date, ROUND(LAG(total_amount) OVER (ORDER BY sale_date),2) AS amount
4   FROM `sprint3_gold.mv_daily_sales`
5 )
6 SELECT
7   ds.sale_date,
8   ROUND(ds.total_amount,2) AS current_day,
9   pd.amount AS previous_day,
10  ROUND(((ds.total_amount - pd.amount)/pd.amount)*100,2) AS percentage_diff
11 FROM `sprint3_gold.mv_daily_sales` AS ds
12 INNER JOIN amount_previous_day AS pd ON pd.sale_date = ds.sale_date
13 ORDER BY ds.sale_date;
14

```

✓ Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo		Resultados	Visualización	JSON	Detalles de la ejecución
Fila	sale_date	current_day	previous_day	percentage_diff	
1	2026-04-22	255210.76	null	null	
2	2026-04-23	532051.78	255210.76	108.48	
3	2026-04-24	532944.0	532051.78	0.17	
4	2026-04-25	511811.3	532944.0	-3.97	
5	2026-04-26	528428.1	511811.3	3.25	
6	2026-04-27	517111.09	528428.1	-2.14	

Bytes procesados

816 B

Bytes facturados

10 MB

Observacions: utilitzem una **CTE** per trobar l'import del dia anterior a través de al funció de finestra **LAG**. A la primera fila el valor és **null** ja que no hi ha fila anterior. Observem que la consulta processa pocs bytes (816B). Això és degut al fet que estem executant una consulta sobre una **Vista Materialitzada**.

Exercici 3: Totals Acumulats (Running Totals sobre Vistes)

Escenari:

El **Director Financer (CFO)** necessita visualitzar la corba de creixement anual. No li interessa tant el que es va vendre avui, sinó quant portem facturat en total des de l'inici de l'any fins avui (**YTD - Year To Date**).

🔧 Tasca:

Utilitzant la vista materialitzada **mv_daily_sales** (per no recalculer agregacions base), genera un informe amb tres columnes:

- » **1.Data.**
- » **2.Vendes del Dia:** Arrodonit a 2 decimals.
- » **3.Vendes Acumulades YTD:** Una suma progressiva de les vendes que ha de reiniciar-se cada **1 de Gener**. El resultat final ha d'estar arrodonit a 2 decimals.

💡 Pista Tècnica:

Per aconseguir el reinici anual, necessites dividir la finestra usant **PARTITION BY EXTRACT(YEAR FROM data)**.
L'estructura completa seria: `SUM(...) OVER (PARTITION BY ... ORDER BY ... ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`.

» Informe Year to Date

```

1
2 SELECT
3   sale_date,
4   ROUND(total_amount,2) AS current_day,
5   ROUND(SUM(total_amount) OVER (
6     PARTITION BY EXTRACT(YEAR FROM sale_date)
7     ORDER BY sale_date
8     ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
9   ),2) AS accumulated_amount
10  FROM `sprint3_gold.mv_daily_sales`
11  ORDER BY sale_date;
12
13

```

✓ Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo	Resultados	Visualización	JSON	Detalles de la ejecución	G
Fila	sale_date	current_day	accumulated_am...		
1	2026-04-22	255210.76	255210.76		
2	2026-04-23	532051.78	787262.54		
3	2026-04-24	532944.0	1320206.54		
4	2026-04-25	511811.3	1832017.84		
5	2026-04-26	528428.1	2360445.94		
6	2026-04-27	517111.09	2877557.03		
7	2026-04-28	502699.11	3380256.14		
8	2026-04-29	517482.27	3897738.41		

Bytes procesados 816 B

Bytes facturados 10 MB

Observacions: Per fer una columna d'acumulats utilitzem **funcions de finestra**.

Sumem els imports dels registres sobre un conjunt de dades (**SUM (total_amount) OVER(...)**)on:

- agrupem les dades per anys de manera que al principi de cada any posem la suma a 0 (**PARTITION BY...**)
- ordenem per dates (**ORDER BY...**)
- seleccionem els registres anteriors i l'actual (**ROWS BETWEEN...**)

Exercici 4: Fidelització i Valor del Client (Filtratge Avançat)

Escenari:

L'equip de **Màrqueting** llança la campanya "**A la tercera va la vençuda**". Volen enviar un regal als usuaris que hagin completat la seva tercera transacció. No obstant això, el pressupost del regal no és fix: depèn de la "qualitat" del client en els seus inicis. Necessiten saber el Tiquet Mitjà d'aquestes 3 primeres compres per segmentar si envien un detall bàsic o un regal premium.

Tasca:

Genera un llistat dels usuaris que han arribat (o superat) la seva tercera compra, mostrant:

- » 1. **Dades d'usuari** (**user_id**, **nom_complet**, **email**).
- » 2. **La data i l'import exacte de la 3a compra**.
- » 3. **Mitjana de les 3 primeres**: La mitjana de despesa de les seves transaccions 1, 2 i 3.

Pista Tècnica:

- » **Has d'utilitzar la funció ROW_NUMBER() per establir l'ordre cronològic de les compres**.
- » **Has d'utilitzar la clàusula QUALIFY per filtrar**.
- » **Repte d'Optimització**: Busca l'estratègia de menor cost computacional. Tingues en compte que calcular mitjanes sobre tot l'historial històric és costós. Intenta filtrar les files necessàries abans de realitzar l'agregació final.

» Informe “A la tercera va la vençuda”

```

2 WITH
3 purchase_rank AS (
4   SELECT user_id, DATE(timestamp) as date, amount, ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY timestamp) AS num_rank
5   FROM `sprint3_gold.fact_transactions_optimized`
6   WHERE declined = 0
7   QUALIFY num_rank <=3
8 ),
9 purchase_average AS(
10  SELECT user_id, ROUND(AVG(amount),2) AS average
11  FROM purchase_rank
12  GROUP BY user_id
13 )
14 SELECT u.user_id, CONCAT(u.name, ' ', u.surname) AS name, u.email, pr.date AS date3rd, pr.amount AS amount3rd, pa.average
15 FROM purchase_rank AS pr
16 INNER JOIN `sprint3_silver.users_combined` AS u USING (user_id)
17 INNER JOIN purchase_average AS pa USING (user_id)
18 WHERE pr.num_rank = 3
19 ORDER BY pa.average DESC;
20

```

Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Chat

Guardar los resultados ▾

Información del trabajo		Resultados	Visualización	JSON	Detalles de la ejecución		Gráfico de ejecución	
fila	user_id	name	email	date3rd	amount3rd	average		
1	349	Yxxcj Fnceqbixmt	yxxcj.fnceqbixmt@example.com	2026-04-24	653.67	611.3		
2	4053	Yieyga Szmwdfhu	yieyga.szmwdfhu@example.com	2026-05-02	506.77	594.43		
3	455	Fwfl Phylgnmhnd	fwfl.phylgnmhnd@example.com	2026-04-23	388.11	581.78		
4	416	Slet Irxec	slet.irxec@example.com	2026-04-25	269.54	567.74		
5	4763	Kvuscq Qtnrhzbz	kvuscq.qtnrhzbz@example.com	2026-04-27	618.71	560.45		
6	2341	Edftrl Hxplqnuq	edftrl.hxplqnuq@example.com	2026-04-25	526.65	552.28		

Bytes procesados 3.31 MB

Bytes facturados 20 MB

Observacions: utilitzem una **CTE** (Common Table Expressions) amb les següents subconsultes:

- **purchase_rank:** agrupa els registres de la taula transactions per usuari, els ordena per data en ordre ascendent i assigna un número enter seqüencial a cada fila. Això ho fem amb la funció de finestra **rownumber()**. Seleccionem els tres primers registres de cada usuari (**QUALIFY num_rank<=3**). D'aquesta manera obtenim les tres primeres compres de cada usuari.
- **purchase_average:** per cada usuari de la subconsulta **purchase_rank** calculem la mitjana de l'import. Com que les dades ja estan filtrades el cost de calcular la mitjana és menor.

Nivell 3: Analytics Engineering (Arrays & Automatització)

Escenari:

Màrqueting necessita urgentment un Rànquing de Productes més Venuts.

Exercici 1: Aplanament de Dades (Unnesting)

Escenari:

Actualment, a la taula de transaccions, els productes venuts estan "amagats" dins d'un Array (una llista d'IDs). L'equip de vendes vol analitzar quins productes específics es venen més, però no poden fer-ho si les dades estan comprimides en una llista. Necessiten una **Taula Detallada (Flat Table)** on cada fila representi un producte venut, encara que això signifiqui que l'ID de la transacció es repeteixi.



Tasca:

Crea la taula **dim_transactions_flat** desnormalitzant la informació. Has d'"explotar" l'array de productes i creuar-lo amb el catàleg mestre per obtenir els noms i preus individuals.



Entrega:

- » Una captura de la Vista Prèvia de la teva taula on es vegi el DDL executada correctament.
- » Una captura de pantalla on es vegi l'esquema de la nova taula.

» Creació de la taula dim_transactions_flat

```
1
2 CREATE OR REPLACE TABLE `sprint3_gold.dim_transactions_flat` AS
3 SELECT
4     t.transaction_id,
5     t.timestamp,
6     t.amount AS total_ticket,
7     p.product_id AS product_sku,
8     p.name AS product_name,
9     p.price AS product_price
10 FROM `sprint3_gold.fact_transactions_optimized` AS t
11 CROSS JOIN UNNEST(t.product_ids) AS product_id
12 INNER JOIN `sprint3_silver.products_clean` as p ON product_id = p.product_id
13 WHERE t.declined = 0
14 ORDER BY t.transaction_id;
15
```

✓ Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo

Resultados

Detalles de la ejecución

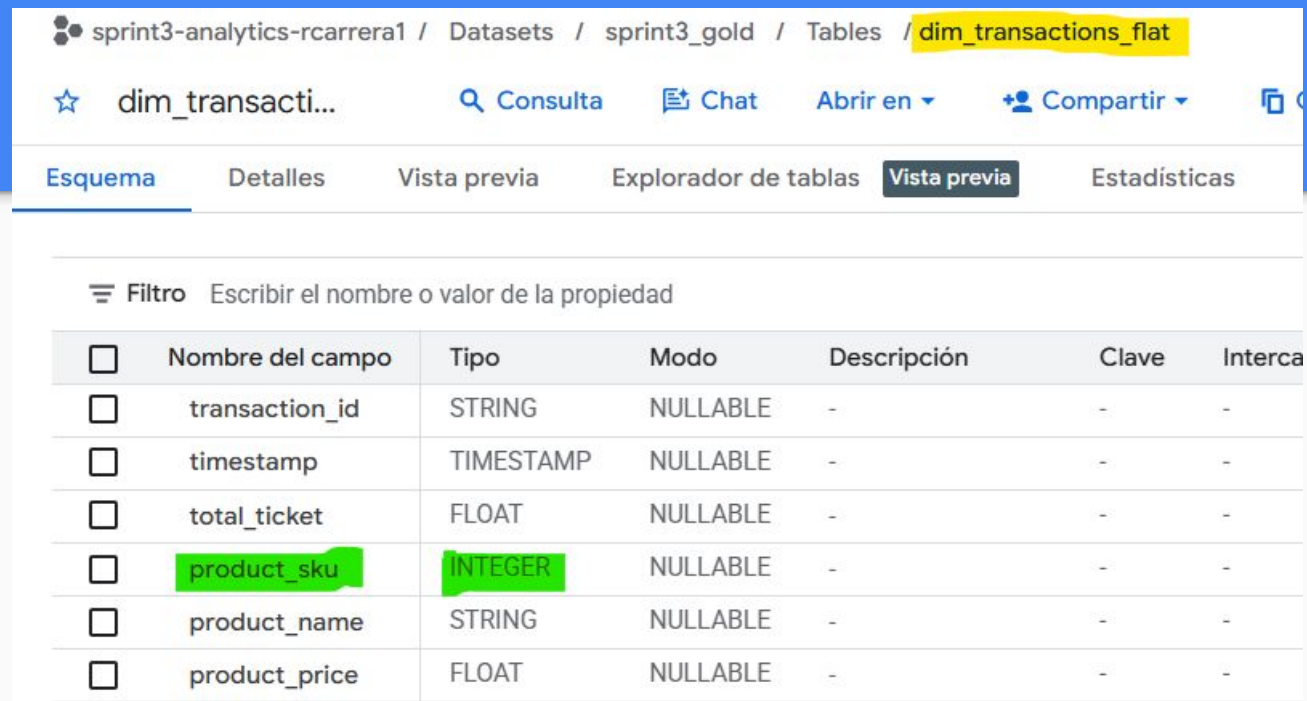
Gráfico de ejecución



Esta declaración reemplazó la tabla denominada dim_transactions_flat.

Observacions: **CROSS JOIN UNNEST (t.product_ids):** genera una fila per cadascun dels elements de l'array que conté el camp **product_ids**.

» Esquema de la taula dim_transactions_flat



sprint3-analytics-rcarrera1 / Datasets / sprint3_gold / Tables / dim_transactions_flat

☆ dim_transacti... Consulta Chat Abrir en ▾ + Compartir ▾

Esquema Detalles Vista previa Explorador de tablas Vista previa Estadísticas

Filtro Escribir el nombre o valor de la propiedad

☐	Nombre del campo	Tipo	Modo	Descripción	Clave	Interca
☐	transaction_id	STRING	NULLABLE	-	-	-
☐	timestamp	TIMESTAMP	NULLABLE	-	-	-
☐	total_ticket	FLOAT	NULLABLE	-	-	-
☐	product_sku	INTEGER	NULLABLE	-	-	-
☐	product_name	STRING	NULLABLE	-	-	-
☐	product_price	FLOAT	NULLABLE	-	-	-

Observacions: a l'esquema de la taula **dim_transactions_flat** observem que el camp **product_sku** és un enter simple. Ja no és un array d'enters.

» Vista prèvia de la taula dim_transactions_flat

sprint3-analytics-rcarrera1 / Datasets / sprint3_gold / Tables / **dim_transactions_flat**

☆ dim_transacti... [Consulta](#) [Chat](#) [Abrir en ▾](#) [Compartir ▾](#) [Copiar](#) [Instantánea](#) [Borrar](#) [↑](#)

Esquema Detalles Vista previa Explorador de tablas **Vista previa** Estadísticas Linaje Perfil de datos Calidad de los datos

Fila	transaction_id	timestamp	total_ticket	product_sku	product_name	product_price
1	00043A49-2949-494B-A5DD-A5BAE3BB19DD	2026-04-28 08:28:59.327163 UTC	395.43	97	jinn Winterfell	65.25
2	00043A49-2949-494B-A5DD-A5BAE3BB19DD	2026-04-28 08:28:59.327163 UTC	395.43	26	Stark Karstark	53.01
3	00043A49-2949-494B-A5DD-A5BAE3BB19DD	2026-04-28 08:28:59.327163 UTC	395.43	87	sith Jade	96.26
4	00043A49-2949-494B-A5DD-A5BAE3BB19DD	2026-04-28 08:28:59.327163 UTC	395.43	16	the duel warden	180.91
5	000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	2026-05-08 08:28:59.327163 UTC	155.63	66	mustafar jinn	2.12
6	000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	2026-05-08 08:28:59.327163 UTC	155.63	69	Littlefinger the giantsblood	57.25
7	000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	2026-05-08 08:28:59.327163 UTC	155.63	87	sith Jade	96.26
8	00045D6B-ED2E-4F2F-8186-CEE074D875D0	2026-05-09 08:28:59.327163 UTC	326.01	11	Karstark Dorne	49.7
9	00045D6B-ED2E-4F2F-8186-CEE074D875D0	2026-05-09 08:28:59.327163 UTC	326.01	30	Karstark warden	79.53
10	00045D6B-ED2E-4F2F-8186-CEE074D875D0	2026-05-09 08:28:59.327163 UTC	326.01	16	the duel warden	180.91
11	00045D6B-ED2E-4F2F-8186-CEE074D875D0	2026-05-09 08:28:59.327163 UTC	326.01	81	the duel	15.87
12	000481C3-1C26-4FEF-83A0-4CD0EB004BBD	2026-05-10 08:28:59.327163 UTC	161.6	72	north	161.6
13	00051AA4-9CBE-4268-B070-C38062A1B3E2	2026-05-08 08:28:59.327163 UTC	148.91	18	Karstark warden	148.91

Observacions: a la **Vista Prèvia** de la taula **dim_transactions_flat** observem que s'ha creat una fila per cadascun dels codis de producte que hi havia a l'array del camp **Product_ids** de la transacció.

Exercici 2: El Rànquing de Vendes (Agregació Simple)

Escenari:

Ara que has preparat la infraestructura i disposes de la taula **dim_transactions_flat** (on cada fila equival a un producte venut), generar informes analítics és trivial. Ja no necessites funcions complexes de desanidament per respondre preguntes bàsiques de negoci.



Tasca:

Genera el **Top 5 de productes** més venuts en la història de la companyia.



Mètode:

Consulta directament la teva nova taula desnormalitzada (dim_transactions_flat).

- » En ser la taula plana, simplement necessites una agrupació estàndard (GROUP BY) pel nom del producte i un compte (COUNT) de les files.



Entrega:

- » El codi SQL que genera el rànkning.
- » Una captura del resultat amb els 5 productes guanyadors i quantes unitats s'han venut de cadascun.

» Informe Top 5 productes més venuts

```
1  
2 SELECT product_name, COUNT (*) AS Quantity  
3 FROM `sprint3_gold.dim_transactions_flat`  
4 GROUP BY product_name  
5 ORDER BY Quantity DESC LIMIT 5;  
6  
7
```

✓ Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo		Resultados	Visualización	JSON	Detalles de
Fila	product_name ▼	Quantity ▼			
1	Winterfell	10041			
2	the duel	10034			
3	dooku solo	10005			
4	Karstark Dorne	7666			
5	skywalker ewok	7582			

Observacions: el fet de tenir la taula **dim_transactions_flat** on tenim una fila per cada producte ens facilita molt la tasca de fer consultes relacionades amb els productes.

Exercici 3: Automatització del Pipeline i Visualització

Escenari:

L'informe estàtic de l'exercici anterior ha agradat molt, però el **Director de Màrqueting** necessita professionalitzar-lo per a la presa de decisions diària. Té dos requeriments urgents:

- » **1. Lògica Centralitzada:** Volen veure el preu amb IVA (21%) desglossat. No obstant això, com que l'impost pot canviar per país o any, et prohibeixen posar el càlcul $\text{preu} * 1,21$ directament en el codi ("Hardcoded"). Has d'utilitzar una funció reutilitzable.
- » **2. Frescor de Dades:** El tauler ha d'estar actualitzat automàticament cada matí a les 07:00 AM, abans de la reunió diària de vendes.



Tasca:

Has d'evolucionar la teva taula **dim_transactions_flat** per incloure el càlcul d'impostos i automatitzar la seva regeneració.



Entrega:

- » El codi SQL de la **UDF** (User Defined Functions).
- » El codi SQL actualitzat de la creació de la taula.
- » Una captura de pantalla demostrant que la **Scheduled Query** està programada.
- » Un enllaç del gràfic a **Looker Studio**.

» 1. User Defined Functions (UDF)

Crea una funció SQL persistent anomenada `calculate_tax(amount)` que rebí un valor numèric i retorni el resultat aplicant el 21% d'impost.

```
1  
2 CREATE OR REPLACE FUNCTION `sprint3_gold.calculate_tax`(amount FLOAT64, tax FLOAT64)  
3 RETURNS FLOAT64  
4 AS (  
5   ROUND (amount + (amount * tax)/100 ,2)  
6 );  
7  
8  
9
```

Se completó la consulta

Resultados de la consulta

Información del trabajo

Resultados

Detalles de la ejecución

Gráfico de ejecución



Esta declaración reemplazó la función llamada `sprint3-analytics-rcarrera1.sprint3_gold.calculate_tax`.

- sprint3_gold
 - Rutinas (1)
 - calculate_tax
 - dim_transactions_flat
 - fact_transactions_optimized
 - mv_daily_sales
 - product_sales_ranking
 - v_marketing_kpis

Observacions: Creem una funció amb dos **paràmetres**: **amount** (base imposable) i **tax** (% impost a aplicar). D'aquesta manera aquesta funció ens servirà per diferents països i també si hi ha un canvi en el percentatge d'impost en un mateix país.

» 2. Integració i Orquestració

a) Modifica el codi de creació de la taula (de l'Exercici 1) perquè utilitzi la teva nova funció `calculate_tax` i generi una columna nova: `product_price_tax_inc`.

```
2 CREATE OR REPLACE TABLE `sprint3_gold.dim_transactions_flat` AS
3 SELECT
4     t.transaction_id,
5     t.timestamp,
6     t.amount AS total_ticket,
7     p.product_id AS product_sku,
8     p.name AS product_name,
9     p.price AS product_price,
10    sprint3_gold.calculate_tax(p.price, 21) AS product_price_tax_inc
11 FROM `sprint3_gold.fact_transactions_optimized` AS t
12 CROSS JOIN UNNEST(t.product_ids) AS product_id
13 INNER JOIN `sprint3_silver.products_clean` as p ON product_id = p.product_id
14 WHERE declined = 0
15 ORDER BY t.transaction_id;
16 --
```

Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo Resultados Detalles de la ejecución Gráfico de ejecu

Esta declaración reemplazó la tabla denominada dim_transactions_flat.

Tables / dim_transactions_flat

Abrir en ▾

+ Compartir ▾

Copiar

Instantánea

tablas	Vista previa	Estadísticas	Linaje	Perfil de datos
	product_name	product_price	product_price_tax_inc	
	jinn Winterfell	65.25	78.95	
	Stark Karstark	53.01	64.14	
	sith Jade	96.26	116.47	
	the duel warden	180.91	218.9	
	mustafar jinn	2.12	2.57	

Observacions: Hem modificat el codi per afegir el nou camp. Quan cridem la funció `calculate_tax` li passem com a paràmetres el preu i la taxa. A la **Vista Prèvia** de la taula comprovem que s'ha creat correctament.

» 2. Integració i Orquestració

b) Configura una Scheduled Query a BigQuery perquè aquesta sentència (CTAS*) s'executi automàticament cada dia a les 07:00 AM.

```
Consulta sin título [Ejecutar] [Guardar] [Descargar] [Compartir]
```

```
1 CREATE OR REPLACE TABLE `sprint3_gold.dim_transactions_flat` AS
2 SELECT
3   t.transaction_id,
4   t.timestamp,
5   t.amount AS total_ticket,
6   p.product_id AS product_sku,
7   p.name AS product_name,
8   p.price AS product_price,
9   `sprint3_gold.calculate_tax`(p.price, 21) AS product_price_tax_inc
10 FROM `sprint3_gold.fact_transactions_optimized` AS t
11 CROSS JOIN UNNEST(t.product_ids) AS product_id
12 INNER JOIN `sprint3_silver.products_clean` AS p ON product_id = p.product_id
13 ORDER BY t.transaction_id;
```

Se produjo un error mientras se creaba la consulta programada: Billing is disabled for this project. Please ask the project owner to enable it.

Resultados de la consulta

Información del trabajo Resultados Detalles de la ejecución Gráfico de ejecución

Esta declaración reemplazó la tabla denominada dim_transactions_flat.

Detalles y programación

Nombre de la consulta programada *

Regeneració Taula dim_transactions_flat

Opciones de programación

Frecuencia de repetición *

Días

A la(s) *

7:00

UTC

☒ Comenzar ahora ☐ Comenzar a la hora definida

Fecha de inicio y hora de ejecución

15/6/26, 10:18 a.m.

000



☒ Sin hora de finalización programada

☐ Programar una hora de finalización

Fecha de finalización

000



Programado para ejecutarse todos los días a las 7:00 UTC a partir de ahora.

Observacions: No es pot programar la consulta ja que no tenim activada la facturació pel projecte.

*CTAS: Create Table As Select

» 3.Visualització (BI)

Connecta Looker Studio a la teva taula dim_transactions_flat i crea un Dashboard: "Monitor de Rendiment de Vendes"

Observacions: [enllaç al gràfic de DataStudio.](#)